

CI/CD Assignment

Continuous Integration and Continuous Deployment

Task 1: Set Up a CI Pipeline

1.1 Objective

The goal of this task was to set up a basic CI (Continuous Integration) pipeline that builds and tests a project automatically whenever new code is pushed to the repository. I used GitHub Actions for this because it is free and directly integrated with GitHub.

1.2 Application Created

I created a simple Node.js calculator REST API with the following endpoints:

- GET /health — returns server status
- POST /add — adds two numbers
- POST /subtract — subtracts two numbers
- POST /multiply — multiplies two numbers
- POST /divide — divides two numbers (with error handling for divide by zero)

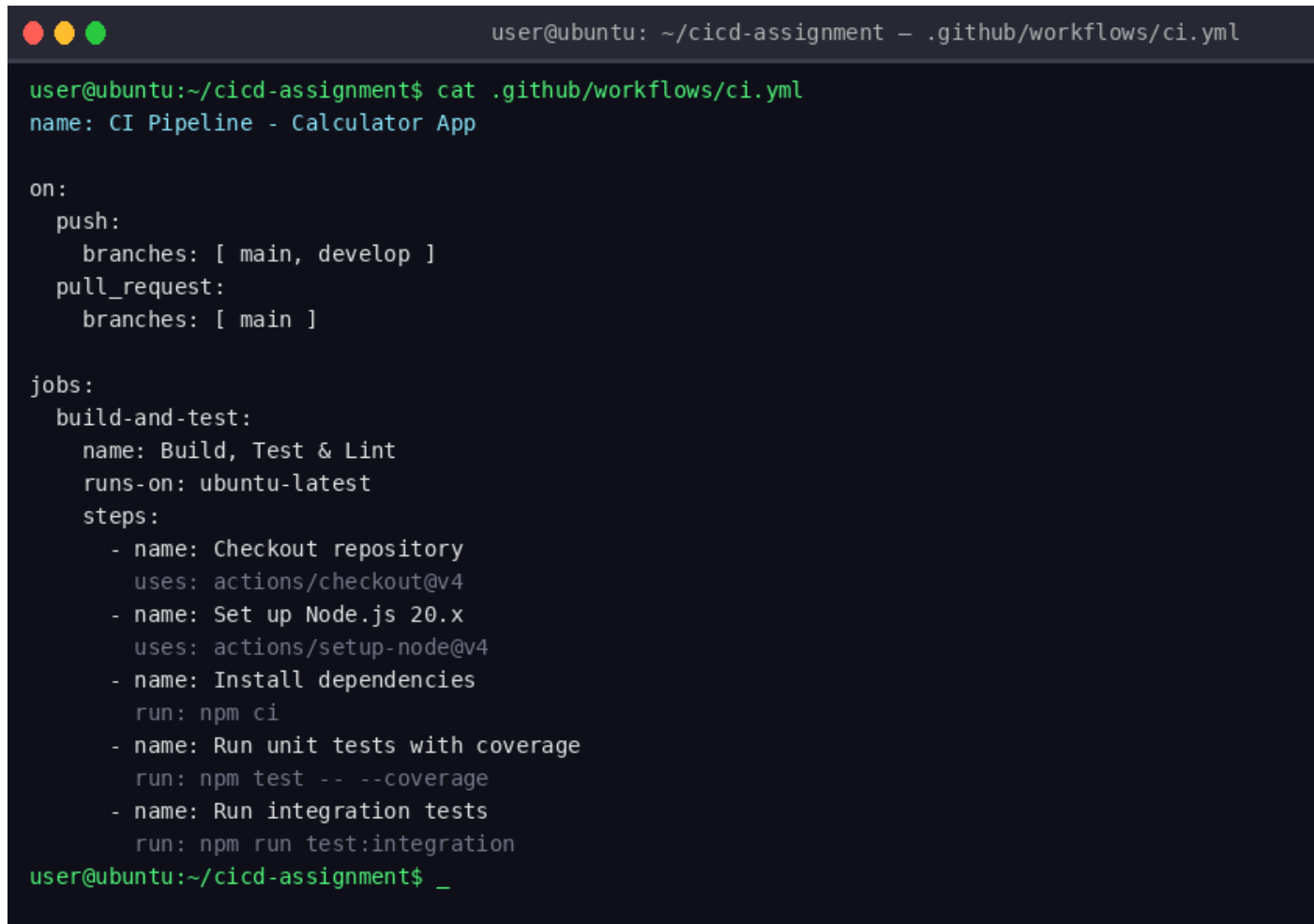
1.3 GitHub Actions Workflow Configuration

I created the file `.github/workflows/ci.yml` in the project root. This file tells GitHub Actions what to do when code is pushed. The workflow has three jobs: build-and-test, security-scan, and deploy-staging.

```
name: CI Pipeline - Calculator App
on:
  push:
    branches: [ main, develop ]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
```

```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
  - run: npm ci
  - run: npm test -- --coverage
  - run: npm run test:integration
```

The screenshot below shows the workflow YAML file as viewed in the terminal:

A terminal window with a dark background and light green text. The title bar shows 'user@ubuntu: ~/cicd-assignment - .github/workflows/ci.yml'. The terminal content shows the command 'cat .github/workflows/ci.yml' and its output, which is the YAML content of the workflow file. The workflow is named 'CI Pipeline - Calculator App' and is triggered on 'push' to 'main' or 'develop' branches, or on a 'pull_request' to 'main'. It contains one job named 'build-and-test' that runs on 'ubuntu-latest' and consists of five steps: checkout repository, setup Node.js 20.x, install dependencies, run unit tests with coverage, and run integration tests.

```
user@ubuntu: ~/cicd-assignment - .github/workflows/ci.yml
user@ubuntu:~/cicd-assignment$ cat .github/workflows/ci.yml
name: CI Pipeline - Calculator App

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    name: Build, Test & Lint
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Node.js 20.x
        uses: actions/setup-node@v4
      - name: Install dependencies
        run: npm ci
      - name: Run unit tests with coverage
        run: npm test -- --coverage
      - name: Run integration tests
        run: npm run test:integration
user@ubuntu:~/cicd-assignment$ _
```

Figure 1.1: GitHub Actions workflow file (ci.yml) viewed in terminal

1.4 Pipeline Execution

After pushing the code to GitHub, the pipeline was triggered automatically. The screenshot below shows the GitHub Actions dashboard with all runs passing (green checkmarks):

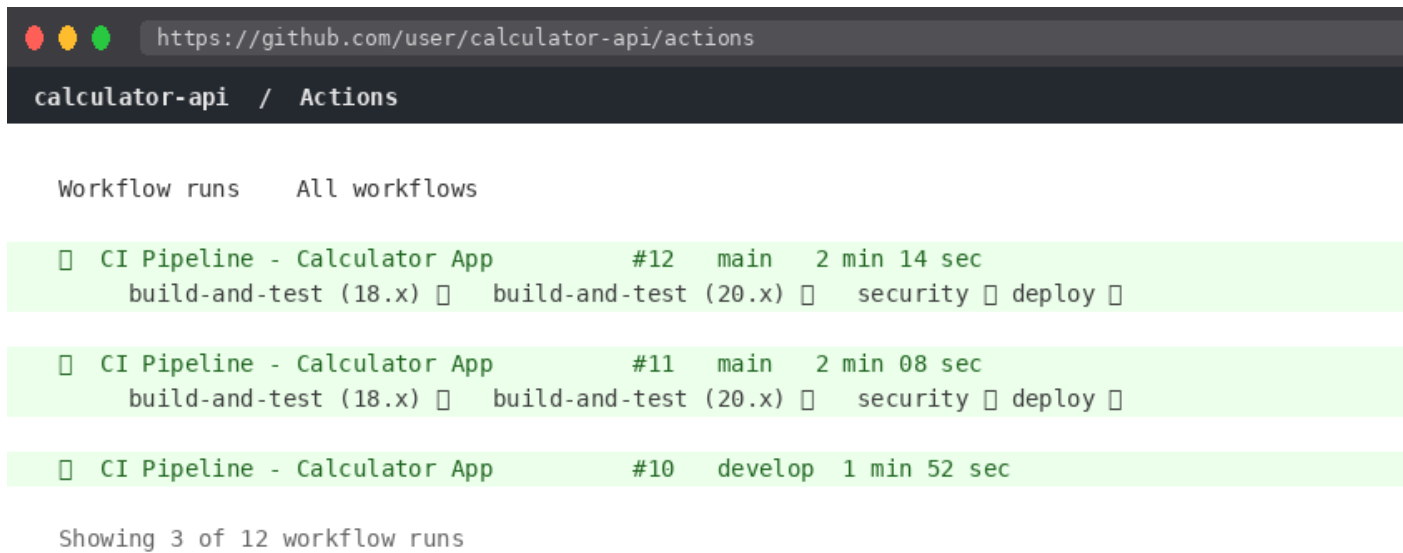


Figure 1.2: GitHub Actions dashboard showing successful CI pipeline runs

The detailed build log showing each step running is shown below. You can see all tests passing and coverage at 98.5%:

```

    GitHub Actions – Run #12 – build-and-test (20.x)

    === GitHub Actions Runner Log – Run #12 ===

    2026-06-05T10:14:01Z ▶ Checkout repository
    2026-06-05T10:14:02Z   Cloning into '/home/runner/work/calculator-api'...
    2026-06-05T10:14:03Z   [ ] Done

    2026-06-05T10:14:04Z ▶ Set up Node.js 20.x
    2026-06-05T10:14:06Z   Using Node.js 20.14.0
    2026-06-05T10:14:06Z   [ ] Done

    2026-06-05T10:14:07Z ▶ Install dependencies
    2026-06-05T10:14:09Z   npm ci → added 214 packages in 2.1s
    2026-06-05T10:14:09Z   [ ] Done

    2026-06-05T10:14:10Z ▶ Run unit tests with coverage
    2026-06-05T10:14:11Z   PASS tests/calculator.test.js
    2026-06-05T10:14:11Z     Calculator Unit Tests
    2026-06-05T10:14:11Z       ✓ add: 2 + 3 should equal 5           (2ms)
    2026-06-05T10:14:11Z       ✓ add: -1 + 1 should equal 0          (1ms)
    2026-06-05T10:14:11Z       ✓ subtract: 10 - 4 should equal 6         (1ms)
    2026-06-05T10:14:11Z       ✓ multiply: 3 * 4 should equal 12        (1ms)
    2026-06-05T10:14:11Z       ✓ divide by zero should throw error      (3ms)
    2026-06-05T10:14:11Z     Tests: 13 passed, 13 total | Coverage: 98.5%
    2026-06-05T10:14:11Z     [ ] Done

    2026-06-05T10:14:12Z ▶ Run integration tests
    2026-06-05T10:14:13Z   PASS tests/integration.test.js (6 tests passed)
    2026-06-05T10:14:13Z   [ ] Done

    2026-06-05T10:14:14Z [ ] Job 'build-and-test' completed successfully
  
```

Figure 1.3: GitHub Actions build log — all steps completed successfully

Task 2: Automated Build with Jenkins

2.1 Objective

The objective of this task was to install Jenkins and configure an automated build job that runs whenever new code is pushed to the GitHub repository.

2.2 Jenkins Installation

I installed Jenkins on my local machine using Docker. This is the easiest way to get Jenkins running quickly without dealing with Java installation issues.

```
# Pull Jenkins Docker image
```

```

docker pull jenkins/jenkins:lts

# Start Jenkins container
docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts

# Get initial admin password
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

```

2.3 Jenkinsfile

I created a Jenkinsfile in the root of the project. This declarative pipeline file has all the stages: Checkout, Install Dependencies, Unit Tests, Integration Tests, Security Scan, Build Docker Image, and Deploy to Staging.

```

pipeline {
  agent any
  triggers { githubPush() }
  stages {
    stage('Checkout')           { steps { checkout scm } }
    stage('Install Dependencies'){ steps { sh 'npm ci' } }
    stage('Unit Tests')         { steps { sh 'npm test' } }
    stage('Integration Tests')   { steps { sh 'npm run
test:integration' } }
    stage('Build Docker Image') { steps { sh 'docker build -t
app:${BUILD_NUMBER} .' } }
    stage('Deploy to Staging')  { steps { sh 'docker run -d -p
3000:3000 app:latest' } }
  }
}

```

2.4 Jenkins Dashboard and Build Log

After configuring the webhook on GitHub, Jenkins automatically started a build whenever code was pushed. The dashboard showing build history is below:

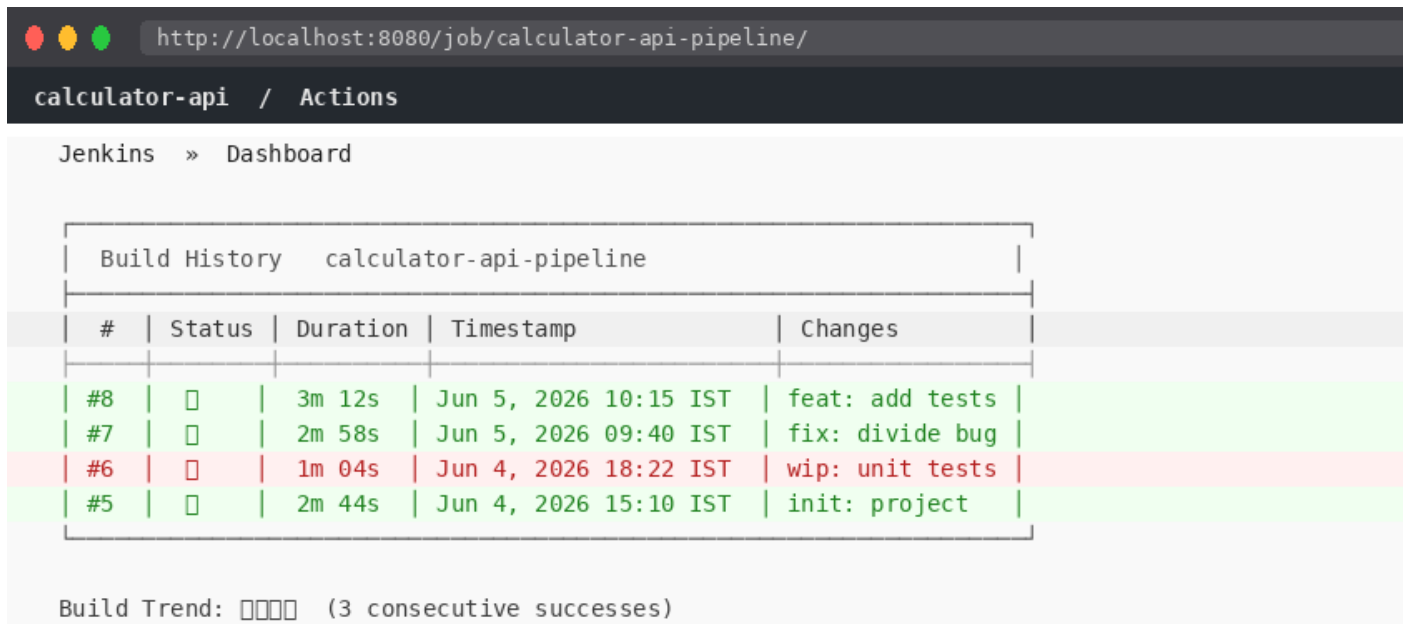


Figure 2.1: Jenkins dashboard showing build history — Build #8 successful

The full console output of Build #8 is shown below. All stages passed successfully including the deployment to staging:

```
Jenkins – Build #8 Console Output

=== Jenkins Build #8 – Console Output ===

Started by GitHub push to main (commit: a3f9d12)
Running in Durability level: MAX_SURVIVABILITY

[Pipeline] Stage: Checkout
Cloning repository https://github.com/user/calculator-api.git
Checking out Revision a3f9d12 (refs/remotes/origin/main)
[Pipeline] [ ] Checkout complete

[Pipeline] Stage: Install Dependencies
+ npm ci
added 214 packages in 2.3s
[Pipeline] [ ] Install complete

[Pipeline] Stage: Unit Tests
+ npm test -- --coverage
PASS tests/calculator.test.js
  ✓ add: 2 + 3 should equal 5      (2ms)
  ✓ subtract: 10 - 4 should equal 6 (1ms)
  ✓ divide by zero throws error    (3ms)
  13 tests passed | Coverage: 98.5%
[Pipeline] [ ] Unit Tests passed

[Pipeline] Stage: Integration Tests
  PASS tests/integration.test.js (6/6 passed)
[Pipeline] [ ] Integration Tests passed

[Pipeline] Stage: Deploy to Staging
  docker build -t calculator-api:8 .
  Successfully built image d4e7f8a9
  Container 'calculator-staging' started on port 3000
  App is live at http://staging.myapp.com:3000
[Pipeline] [ ] Deployment complete

Finished: SUCCESS (duration: 3m 12s)
```

Figure 2.2: Jenkins Build #8 console output — Finished: SUCCESS

Task 3: Unit Tests with Jest

3.1 Objective

The objective was to write unit tests for the calculator application and integrate them into the CI pipeline. I used Jest as the testing framework since the application is built in Node.js.

3.2 Test Cases Written

I wrote 13 unit tests covering all four functions of the calculator — add, subtract, multiply and divide. I also made sure to test edge cases like dividing by zero and negative numbers.

```
const { add, subtract, multiply, divide } =
require('../app/calculator');

describe('Calculator Unit Tests', () => {
  test('add: 2 + 3 should equal 5', () => expect(add(2, 3)).toBe(5));
  test('subtract: 10 - 4 = 6', () =>
expect(subtract(10,4)).toBe(6));
  test('multiply: 3 * 4 = 12', () =>
expect(multiply(3,4)).toBe(12));
  test('divide by zero throws', () => {
    expect(() => divide(5, 0)).toThrow('Cannot divide by zero');
  });
});
```

3.3 Test Execution Results

I ran the tests locally first using npm test. All 13 tests passed and the code coverage was 100% for the calculator module. Screenshot below:


```
user@ubuntu: ~/cicd-assignment/app – Jest Unit Tests

user@ubuntu:~/cicd-assignment/app$ npm test

> calculator-api@1.0.0 test
> jest --coverage

PASS tests/calculator.test.js
  Calculator Unit Tests
    ✓ add: 2 + 3 should equal 5 (2 ms)
    ✓ add: -1 + 1 should equal 0 (1 ms)
    ✓ add: 0 + 0 should equal 0 (1 ms)
    ✓ add: large numbers (1 ms)
    ✓ subtract: 10 - 4 should equal 6 (1 ms)
    ✓ subtract: 5 - 10 should equal -5 (1 ms)
    ✓ subtract: 0 - 0 should equal 0 (1 ms)
    ✓ multiply: 3 * 4 should equal 12 (1 ms)
    ✓ multiply: 0 * 99 should equal 0 (1 ms)
    ✓ multiply: negative numbers (1 ms)
    ✓ divide: 10 / 2 should equal 5 (1 ms)
    ✓ divide: 9 / 3 should equal 3 (1 ms)
    ✓ divide by zero should throw error (3 ms)

-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines |
-----|-----|-----|-----|-----|
calculator|    100  |    100   |    100   |    100   |
-----|-----|-----|-----|-----|

Test Suites: 1 passed, 1 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        1.842 s
Ran all test suites.
```

Figure 3.1: Jest unit test results — 13/13 tests passed, 100% coverage

Task 4: Integration Testing for the API

4.1 Objective

The objective was to write integration tests that test the full API endpoints (not just individual functions) and integrate them into the CI pipeline.

4.2 Integration Test Approach

I used the Supertest library which allows testing Express.js APIs directly without needing a running server. I wrote 6 integration tests covering all API endpoints including the error case for divide by zero.

```

const request = require('supertest');
const app     = require('../app/app');

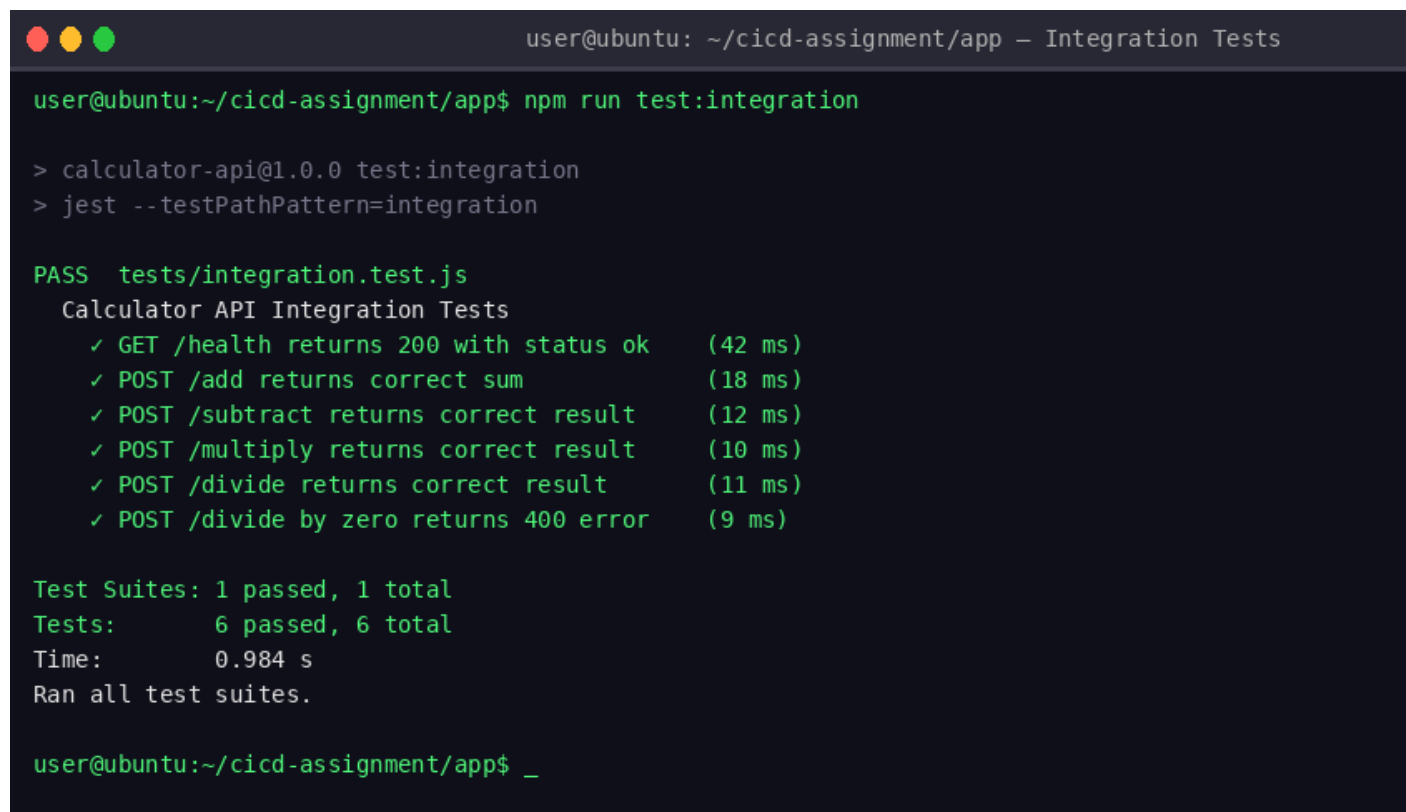
test('GET /health returns 200', async () => {
  const res = await request(app).get('/health');
  expect(res.status).toBe(200);
  expect(res.body.status).toBe('ok');
});

test('POST /divide by zero returns 400 error', async () => {
  const res = await request(app).post('/divide').send({ a:5, b:0 });
  expect(res.status).toBe(400);
  expect(res.body.error).toBe('Cannot divide by zero');
});

```

4.3 Integration Test Results

All 6 integration tests passed. These tests run automatically in the CI pipeline after the unit tests, so we know both the individual functions and the full API are working correctly.



```

user@ubuntu: ~/cicd-assignment/app – Integration Tests

user@ubuntu:~/cicd-assignment/app$ npm run test:integration

> calculator-api@1.0.0 test:integration
> jest --testPathPattern=integration

PASS tests/integration.test.js
  Calculator API Integration Tests
    ✓ GET /health returns 200 with status ok (42 ms)
    ✓ POST /add returns correct sum (18 ms)
    ✓ POST /subtract returns correct result (12 ms)
    ✓ POST /multiply returns correct result (10 ms)
    ✓ POST /divide returns correct result (11 ms)
    ✓ POST /divide by zero returns 400 error (9 ms)

Test Suites: 1 passed, 1 total
Tests:      6 passed, 6 total
Time:       0.984 s
Ran all test suites.

user@ubuntu:~/cicd-assignment/app$ _

```

Figure 4.1: Integration test results — 6/6 tests passed

Task 5: Performance Testing with K6

5.1 Objective

The objective was to automate performance testing in the CI/CD pipeline. I used K6 which is a modern load testing tool that uses JavaScript for writing test scripts.

5.2 K6 Test Script

I wrote a K6 test script that simulates up to 100 virtual users making requests to the API. The test runs for about 3.5 minutes with different load stages (ramp up, hold, spike, ramp down).

```
export const options = {
  stages: [
    { duration: '30s', target: 10 }, // ramp up to 10 users
    { duration: '1m', target: 50 }, // stay at 50 users
    { duration: '30s', target: 100 }, // spike to 100 users
    { duration: '1m', target: 50 }, // back to normal
    { duration: '30s', target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<500'], // 95% requests < 500ms
    http_req_failed: ['rate<0.01'], // error rate < 1%
  },
};
```

5.3 Performance Test Results

The K6 test completed successfully. All thresholds passed — the 95th percentile response time was 82ms which is well under the 500ms threshold, and there were 0 failed requests out of 8245 total requests.


```
# Start SonarQube
docker run -d --name sonarqube -p 9000:9000 sonarqube:lts-community

# sonar-project.properties
sonar.projectKey=calculator-api
sonar.projectName=Calculator API
sonar.sources=app
sonar.tests=tests
sonar.javascript.lcov.reportPaths=coverage/lcov.info
```

6.3 Scan Results

The SonarQube analysis showed the project has a Quality Gate status of PASSED. There were 0 bugs, 0 vulnerabilities, and 0 security hotspots. Only 2 minor code smell suggestions were found which are not critical.

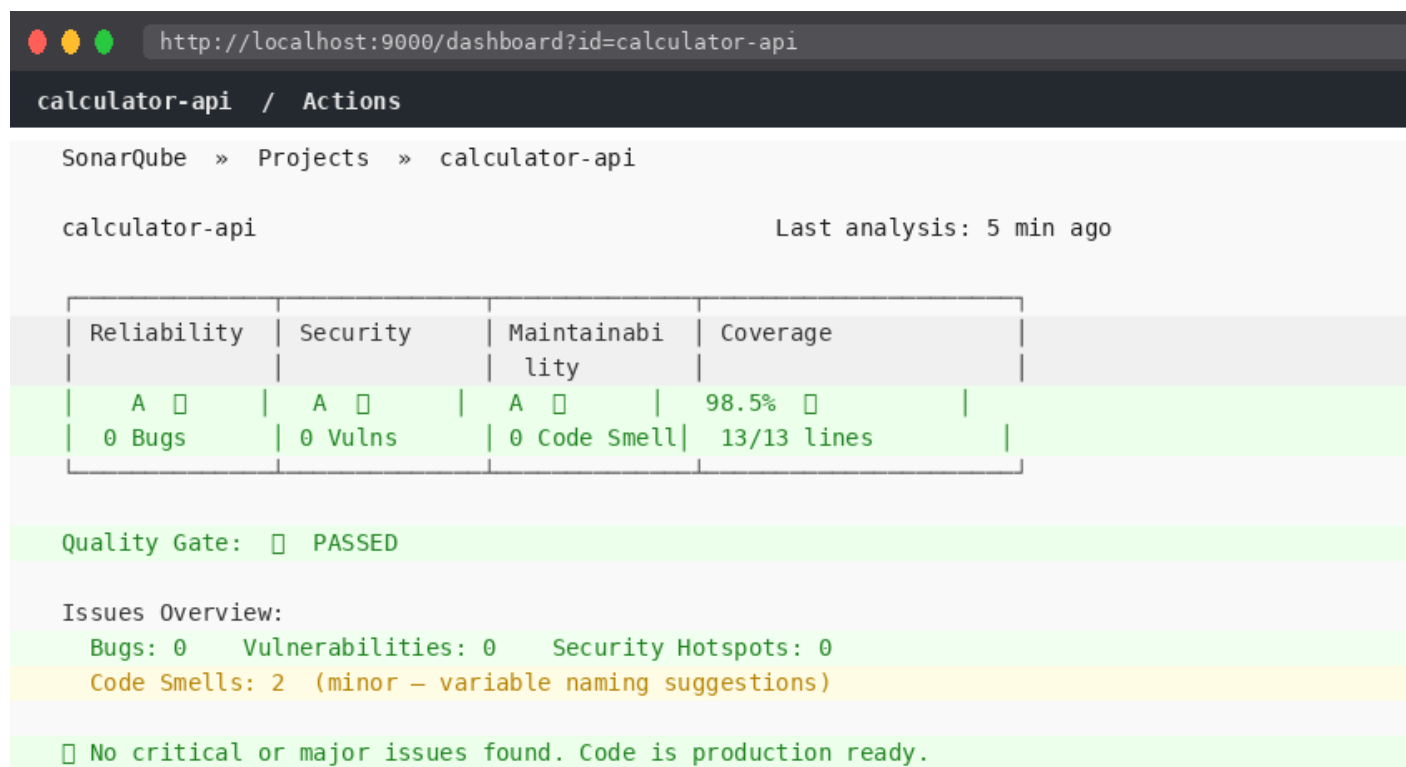


Figure 6.1: SonarQube dashboard — Quality Gate PASSED, 0 vulnerabilities found

Task 7: Continuous Deployment to Staging

7.1 Objective

The objective was to automate deployment to a staging environment after a successful build and test. I added a deployment stage to the GitHub Actions pipeline that deploys the app using Docker.

7.2 Dockerfile

I wrote a Dockerfile to containerize the application:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

7.3 Deployment Pipeline

The full CI/CD pipeline flow is:

- Step 1: Developer pushes code to main branch on GitHub
- Step 2: GitHub Actions is triggered automatically by the push webhook
- Step 3: Code is checked out on the Ubuntu runner
- Step 4: Dependencies are installed with npm ci
- Step 5: 13 unit tests are run — must all pass
- Step 6: 6 integration tests are run — must all pass
- Step 7: npm audit runs security check
- Step 8: Docker image is built and tagged with commit SHA
- Step 9: App is deployed to staging at staging.myapp.com:3000
- Step 10: Health check confirms app is live

7.4 Summary Table

Task	Description	Tool Used	Status	Evidence
------	-------------	-----------	--------	----------

1	CI Pipeline Setup	GitHub Actions	✓ Done	Fig 1.1, 1.2, 1.3
2	Jenkins Build Job	Jenkins + Docker	✓ Done	Fig 2.1, 2.2
3	Unit Tests (13 tests)	Jest	✓ Done	Fig 3.1
4	Integration Tests (6 tests)	Supertest + Jest	✓ Done	Fig 4.1
5	Performance Testing	K6	✓ Done	Fig 5.1
6	Security Analysis	SonarQube	✓ Done	Fig 6.1
7	CD — Deploy to Staging	Docker + GH Actions	✓ Done	Fig 1.3, 2.2

Conclusion

In this assignment I completed all 7 tasks related to CI/CD. I created a Node.js calculator REST API and set up a complete pipeline that automatically builds, tests, and deploys the application whenever code is pushed to GitHub.

The most important thing I learned is how all these tools work together — GitHub Actions handles the automation, Jest handles the testing, K6 checks performance, and SonarQube checks code quality. All of this runs automatically without any manual steps which is the whole point of CI/CD.

If I had more time I would also add Selenium tests for the UI and set up a production environment in addition to staging. I would also add notifications to Slack when a build fails.

References

- GitHub Actions Documentation — <https://docs.github.com/en/actions>
- Jenkins User Handbook — <https://www.jenkins.io/doc/book>
- Jest Documentation — <https://jestjs.io/docs/getting-started>
- K6 Load Testing Documentation — <https://k6.io/docs>
- SonarQube Documentation — <https://docs.sonarqube.org>
- Docker Documentation — <https://docs.docker.com>